Cola de comandos do Lean

Tudo o que você vai digitar no minicurso, em uma página por assunto

Para consultar durante os dois dias e nos exercícios de casa. Todos os exemplos podem ser digitados no Lean Web (`live.lean-lang.org`) logo abaixo do cabeçalho `import Mathlib.Data.Nat.Basic`. **Digite** em vez de copiar do PDF: a cópia costuma estragar espaços e símbolos.

1. Comandos

Comando	O que faz	Exemplo
`import`	Carrega uma biblioteca. Vai na *primeira* linha do arquivo.	`import Mathlib.Data.Nat.Basic`
`#eval`	Calcula o valor de uma expressão.	`#eval 2 + 5` mostra `7`
`#check`	Mostra o tipo de uma expressão (para uma prova, o enunciado).	`#check 2 + 2 = 4` mostra `... : Prop`
`#print`	Mostra uma definição.	`#print Nat`
`def`	Define um objeto ou uma função.	`def dobro (n : ℕ) : ℕ := 2 * n`
`fun x => ...`	Função sem nome ($x \mapsto \ldots$).	`#eval (fun x => x + 1) 3`
`theorem`	Enunciado e prova, com nome (para usar depois).	`theorem t : 2 + 2 = 4 := by rfl`
`example`	Enunciado e prova, sem nome (exercício).	`example : 2 + 2 = 4 := rfl`
`:=`	Separa o enunciado da prova.	`... : enunciado := prova`
`by`	Abre o modo tática: a prova vira uma lista de passos.	`:= by` e as táticas nas linhas de baixo
`sorry`	"Prova pendente". Aviso amarelo: o teorema *não* está provado.	`example ... := by sorry`
`--`	Comentário: o Lean ignora o resto da linha.	`rw [ih] -- usa a HI`

2. Táticas

Escreva uma por linha, depois do `by`, todas na mesma coluna. Ponha o cursor em cada linha para ver a meta mudar no Infoview.

Tática	O que faz	No papel
`rfl`	Fecha `a = b` quando os dois lados são iguais por definição (por cálculo).	"É imediato."
`exact h`	Fecha a meta com `h`, que é exatamente o que se pede.	"...que é a hipótese h."
`apply h`	Com `h : A → B` e meta `B`, troca a meta por `A`.	"Basta provar A."
`intro h`	Com meta `P → Q`, põe `h : P` no contexto; a meta vira `Q`.	"Suponha P."
`rewrite [h]`	Com `h : a = b`, troca `a` por `b` na meta. Não fecha a meta sozinho.	"Por (h), temos..."

Tática	O que faz	No papel
`rw [h]`	Igual a `rewrite [h]`, e no fim tenta `rfl`.	Substituir e ver se acabou.
`rw [← h]`	Usa `h` da direita para a esquerda: troca `b` por `a`.	Ler a igualdade ao contrário.
`rw [h1, h2]`	Várias substituições em sequência.	Uma cadeia de igualdades.
`simp`	Simplifica a meta com lemas automáticos.	"Simplificando…"
`simp [h]`	Igual, usando também `h` (ou um lema) como regra.	"Usando (h) e simplificando."
`induction n with`	Divide em caso base (`| zero =>`) e passo (`| succ n ih =>`).	"Por indução em n."
`cases n with`	Divide em `n = 0` e `n = k + 1`, *sem* hipótese de indução.	"Separemos em casos."
`have h : P := ...`	Prova um fato auxiliar e o põe no contexto com o nome `h`.	"Afirmação: …"
`obtain ⟨x, hx⟩ := h`	Abre um "existe": dá nome à testemunha `x` e à sua propriedade `hx`.	"Seja x tal que…"
`omega`	Resolve sozinho aritmética linear em $\mathbb{N}$ e $\mathbb{Z}$.	"Conta de rotina."
`ring`	Resolve identidades algébricas (precisa de `import Mathlib`).	"Expandindo os dois lados."
`decide`	Decide afirmações com números concretos, como `2 + 2 ≠ 5`.	"Por verificação direta."
`exact?`	Procura um lema que fecha a meta e sugere a linha pronta.	Consultar o índice do livro.

Atenção: nos exercícios de Peano, a graça é *não* usar `omega`, `ring` e `simp`: eles provam sozinhos o que queremos construir passo a passo.

3. Indução: o molde

```
theorem meu_zero_add (n : ℕ) : 0 + n = n := by
  induction n with
  | zero => rfl                    -- caso base:  ⊢ 0 + 0 = 0
  | succ n ih =>                   -- passo:  ih : 0 + n = n
                                   --         ⊢ 0 + (n + 1) = n + 1
    rw [Nat.add_succ]              --         ⊢ (0 + n).succ = n + 1
    rw [ih]                        --         No goals
```

Os nomes `zero` e `succ` são obrigatórios (são os construtores de $\mathbb{N}$); os nomes `n` e `ih` são escolha sua. O Lean mostra o sucessor como `n + 1`.

4. Lemas sobre $\mathbb{N}$ usados no minicurso

Lema	Enunciado
`Nat.add_zero`	`n + 0 = n` (1.ª equação da adição)
`Nat.add_succ`	`n + succ m = succ (n + m)` (2.ª equação da adição)
`Nat.succ_add`	`succ n + m = succ (n + m)`

Lema	Enunciado
`Nat.zero_add`	`0 + n = n`
`Nat.add_comm`	`n + m = m + n`
`Nat.add_assoc`	`n + m + k = n + (m + k)` (`n + m + k` quer dizer `(n + m) + k`)
`Nat.mul_zero`	`n * 0 = 0`
`Nat.mul_succ`	`n * succ m = n * m + n`
`Nat.succ_ne_zero`	`succ n ≠ 0` (axioma 3 de Peano)
`Nat.succ.inj`	`succ m = succ n → m = n` (axioma 4 de Peano)

Para ver o enunciado exato de qualquer lema no Lean Web: `#check Nat.add_succ`. Lemas são *fatos*; táticas são *atos*: `Nat.add_succ` é um lema, `rw [Nat.add_succ]` é uma tática que o usa.

5. Lógica: como montar e usar provas

Enunciado	Para provar	Para usar uma hipótese `h`
`P → Q`	`intro hp` ("suponha P")	`apply h`, ou `h hp` para obter `Q`
`P ∧ Q`	`⟨prova de P, prova de Q⟩`	`h.1 : P` e `h.2 : Q`
`P ∨ Q`	`Or.inl hp` ou `Or.inr hq`	`cases h`
`∃ x, P x`	`⟨testemunha, prova⟩`, ex.: `⟨2, rfl⟩`	`obtain ⟨x, hx⟩ := h`
`a = b`	`rfl`, `rw [...]`	`rw [h]` ou `rw [← h]`
`a ≠ b`	`decide` (números concretos)	—

6. Mensagens que você vai ver

Mensagem	O que quer dizer
`No goals` (verde)	Prova completa.
`unsolved goals`	Faltam passos: a meta mostrada ainda não foi provada.
`declaration uses 'sorry'` (amarelo)	Há um `sorry`: o teorema não está provado.
`Tactic 'rfl' failed`	Os dois lados não são iguais *por definição* (ex.: `0 + n = n`).
`Did not find an occurrence of the pattern`	O `rw` não achou o lado esquerdo do lema na meta.
`Unknown identifier`	Nome que não existe: erro de digitação, ou lema inventado (comum em respostas de IA).
`has already been declared`	Já existe um teorema com esse nome: use outro (ex.: `meu_zero_add`).
`Type mismatch`	Você entregou uma prova de outra coisa (ex.: `exact hpq` quando a meta é `Q`).

7. Símbolos

Símbolo	Digite	Ou escreva	Significa
$\mathbb{N}$	`\N`	`Nat`	os naturais
$\to$	`\to`	`->`	implica / função
$\leftarrow$	`\l`	`<-`	reescrever ao contrário
$\forall$	`\forall`	`forall`	para todo
$\exists$	`\ex`	`exists`	existe
$\wedge$	`\and`		e
$\vee$	`\or`		ou
$\neq$	`\ne`		diferente
$\langle\ \rangle$	`\<` e `\>`		monta pares e testemunhas
$\vdash$	(só aparece no Infoview)		"falta provar que"

Digite o atalho e aperte espaço. No Lean Web, passar o mouse sobre um símbolo mostra como digitá-lo.

No Natural Number Game os nomes não têm `Nat.` (`add_zero`, `add_succ`), a indução se escreve `induction n with d hd`, o sucessor aparece como `succ d` e o `rw` não fecha a meta sozinho: termine com `rfl`.